

Question 1:

For part 1, we generated x uniformly between the bounds of the integral, 0 and 1, because we are integrating with respect to the Lebesgue measure, which is the probability distribution of a uniform random variable. We then transformed these generated x into the integrand $(1 - x^2)^{3/2}$. By the mean value theorem, the average of these generated integrands multiplied by the length of the integral bounds $1 - 0 = 1$ gives us an approximation of the integral's value.

For part 2, there does not exist a uniform random variable on $(0, \infty)$ *at least with respect to the Lebesgue measure!*, so we need a different solution. Noticing that

$$\int_0^\infty \int_0^x e^{-(x+y)} dy dx = \int_0^\infty e^{-x} \int_0^x e^{-y} dy dx = \mathbb{E}_g\left(\int_0^x e^{-y} dy\right) \approx_{LLN} \frac{1}{N} \sum_{i=1}^N \int_0^{x_i} e^{-y} dy$$

where x_i have PDF $g(x) = e^{-x}$ for $x \geq 0$, we can generate e^{-x} uniformly *thanks again to the Lebesgue measure* and back out our exponentially distributed x_i to use in calculating $\int_0^{x_i} e^{-y} dy$ as in step 1, which is then used with the law of large numbers to approximate the original integral.

Question 2:

For part 1 we were asked to test the Central Limit Theorem for a sample mean and sample standard deviation, constructing sample confidence intervals, and determining the probability that our sampled values lie within the interval. By using a Monte Carlo simulation with 10^4 repetitions we found this probability to be .9497 which is approximately equal to our expected probability of 0.95 from the CLT.

For part 2, we did the same thing but for an exponential distribution with $\lambda = 1$. The estimated probability for this was .9503 which is still very close to the theoretical value of .95. By doing these two simulations, we can confirm the accuracy of the CLT for large samples such as 10^4 .

Question 3:

For part 1, we need to find the best constant $c\lambda$ for the AR method. Taking $\exp\lambda$ as our $g(x)$ function, we calculate $\frac{f(x)}{g(x)}$, then take the derivative to find the maximum value that this function can take. The derivative = 0 when $x = 0$ *minimum possible value of x* or when $x^* = \frac{2}{1-\lambda}$, which is our maximum value. Therefore, $\frac{f(x)}{g(x)}$ must be less than or equal to its evaluation at x^* , and we get $c\lambda = \frac{f(x^*)}{g(x^*)} = \frac{2e^{-2}}{\lambda(1-\lambda)^2}$

For part 2, we want to make the AR method the most efficient as we can: a uniform RV gets accepted if it's less than $\frac{f(X)}{cg(X)}$. The probability of acceptance is the expected value of this formula, which comes out to be $\frac{1}{c}$. Therefore, if we want our probability of acceptance to be the highest possible value, we need to minimize c , to maximize the acceptance rate. If we take the derivative of our $c\lambda$ function, set it equal to 0, we get $\lambda^* = 1/3$. From a boundary check, we verify that this is a minimum, and thus our optimal λ^*

For part 3, we first defined c with our optimal λ^* , $f(x)$, and $g(x)$, then calculated the true Expected Value of $f(x)$ to be $\int_0^\infty x \cdot \frac{1}{2}x^2e^{-x}dx = 3$. Using this true mean, we can compare our sample means for differing numbers of samples to see when we start to experience diminishing returns.

The algorithm runs a for loop for each value of $n = 10^1, 10^2, \dots 10^7$. In each case we use AR method, taking only accepted samples into a list, then computing our sample mean.

We seem to begin experiencing diminishing returns around $n = 10^5$, where our sample mean is 3.0077, and $n = 10^6$'s sample mean is 2.9985. We start to gain only small amounts of accuracy in a tradeoff for exceptionally long computation times, with $n = 10^7$ producing sample mean of 3.0003 in return for about 3 full minutes of computation.

Question 4:

In part 1, we were asked to find the expected value of the given N by a simulation. By taking the natural log of each side, we are able to simplify the problem to $\sum_{i=1}^n X_i \leq 5$ where $X_i = -\ln(U_i) \sim \text{Exp}(1)$. From this we derive that $N \sim \text{Poi}(5)$ and thus E

$$N$$

= 5. We then simulate exponential RVs with $\lambda = 1$, recording how many progressive simulated values can sum before breaking the barrier of 5. Then we take the EV of our 10^4 recordings. We found the expected value of N to be 4.9924, which is almost exactly equal to the true mean of 5.

In part 2 of the question, we simply run the same simulation, seeing what proportion of our recorded values for N happen to be 1, 2, ... 20. Our results are shown below:

k | Simulated | Poisson $\lambda = 5$

0 0.0065 0.0067

1 0.0331 0.0337

2 0.0844 0.0842

3 0.1424 0.1404

4 0.1763 0.1755

5 0.1735 0.1755

6 0.1478 0.1462

7 0.1044 0.1044

8 0.0648 0.0653

9 0.0355 0.0363

10 0.0178 0.0181

11 0.0081 0.0082

12 0.0032 0.0034

13 0.0015 0.0013

14 0.0004 0.0005

15 0.0002 0.0002

16 0.0000 0.0000

17 0.0000 0.0000

18 0.0000 0.0000

19 0.0000 0.0000

20 0.0000 0.0000

In end, our best guess for λ is indeed 5, since from our simulation, we get a sample mean of ~ 5 , and the EV of a Poisson distribution is equal to λ

Question 5:

For each policy holder we uniformly generated a number between 0 and 1. If that number was less than 0.05, we uniformly generated another number between 0 and 1 which we transformed into an exponentially distributed number and added that amount to the total claim dollar amount for that simulation. After iterating over every policy holder, if the total claims exceeded \$5000, we added it to the count of 'successful' simulations. We did this $N = 10^4$ many times because any higher order of magnitude took too long, and found the proportion of times the claims exceeded the threshold was $p = 0.1051$.

Question 6:

In part 1, we want to simulate the log return between the initial stock price and the price at $T = 1$, for differing values of N , where N represents the number of steps *changes in price* over the year. We can use the binomial model to simulate the number of successes vs. failures in N steps, where successes represent upward moves, defined using the given u , and failures represent downward moves, defined with $1/u$. From there we simply multiply the initial price by u for the amount of successes, and vice versa for d . We run 1,000,000 simulations, each using 10,000 steps, and calculate our sample mean and standard deviation.

We run these 1,000,000 simulations 4 separate times, each with differing values of μ to check the dependence on μ . Plotting histograms for each of the 4 cases, with a $N(r - \sigma^2/2, \sigma)$ distribution overlayed to check accuracy, we verify that for a high number of simulations, the log returns do indeed converge to that distribution. Furthermore, different values of μ do not change this convergence, so we can infer that there is no dependence on μ . Our simulated EVs and SDs are below:

$\mu = -0.2000$:

Estimated mean: -0.029071,

Estimated std: 0.397910

$\mu = 0.0000$:

Estimated mean: -0.030370,

Estimated std: 0.400290

$\mu = 0.2000$:

Estimated mean: -0.030101,

Estimated std: 0.402073

$\mu = -0.0300$

Estimated mean: -0.029231,

Estimated std: 0.399911

For part 2, we approximate the price of a European call using similar simulation, and compare it to the calculated Black-Scholes price for different values of M . We do this 4 times for each different value of μ . We simulate the price after 1 year in the same way as part 1, using 10,000 steps, approximate the EV of the call using the sample mean, and see if the approximations converge to the Black-Scholes price as the number of simulations increases.

We plot 4 graphs showing the Black-Scholes price as a red dashed line, and the approximations alongside it. We see that as n increases, the approximation approaches the Black-Scholes price, with error decreasing along the way. We see that price converges at almost exactly the same rate regardless of the value of μ , confirming that neither the approximation nor the rate of convergence depend on μ